ARE-II Ready Package

Proposal Skeleton, Method and Evaluation Protocol, Pilot Evidence,
Related Work Map, and ARE-II Execution Plan

CISC-727 — Advanced Research Explorations (ARE) - I
Assignment A05

Kenneth Peter Fernandes
Department of Computer and Information Sciences
Harrisburg University of Science and Technology

Instructor: Prof. Majid Shaalan, Ph.D.

April 25, 2026

Contents

Abstract 4

Part I — Proposal Skeleton 5

1 Problem Statement 5

2 Research Gap and Novelty Paragraph 5

3 Research Questions and Hypotheses 6

3.1 Primary research question 6

3.2 Falsifiable hypotheses 6

4 Proposed Contributions 7

5 Method Overview 7

6 Evaluation Plan Summary 7

7 Risks and Mitigations 8

8 ARE-II Execution Plan 8

Part II — Related Work Package 10

9 Literature Map 10

10 Gap Matrix 10

11 Annotated Bibliography 10

12 Closest Competitor and How This Work Differs 13

Part III — Method and Evaluation Protocol 14

13 System Under Test 14

14 Workload, Model, and Data 14

15 Partitioning, Communication, Agglomeration, and Mapping (PCAM) 14

16 Predicted Bottlenecks (Work-Depth Reasoning) 15

17 Baselines 15

18 Metrics 15

19 Ablation Plan 16

20 Measurement Protocol and Fairness Policy 16

21 Threats to Validity 16

22 Success Criteria 16

Part IV — Pilot Evidence 17

23 Baseline Reproduction Summary 17

24 Strong Result (Primary Figure) 17

25 Interpretation of the Strong Result 17

26 Protocol Compliance Note 18

Part V — Reproducibility 20

27 Reproduction Instructions 20

A AI Usage Log 21

Abstract

This package consolidates the work of CISC-727 (ARE-I) into a single, advisor-ready research bundle and proposes the next-semester (CISC-733, ARE-II) execution plan. The semester opened with a research interest in making long-context Large Language Model inference cheaper through IO-aware attention kernels and Key-Value cache management on commodity GPUs. The literature review (A02) narrowed this to a controlled question that is feasible on a single GPU: *at what sequence-length and hardware conditions does an exact IO-aware attention kernel deliver meaningful efficiency gains over standard attention?* The pilot work (A03 and A04) used a Vision Transformer on CIFAR-10 and a Tesla T4 (Turing-class) GPU on Kaggle, sweeping partition granularity to vary sequence length from 5 to 65 tokens. The pilot reported a null-to-negative result: FlashAttention showed near-zero speedup at 5 tokens and was approximately sixteen percent slower than standard attention at 65 tokens. Late-stage investigation, captured in this package, revealed an important confound: PyTorch's `SDPBackend.FLASH_ATTENTION` path requires Streaming Multiprocessor capability 8.0 or newer (Ampere or newer), and the Tesla T4 is Streaming Multiprocessor capability 7.5 (Turing). The pilot likely measured a silent fallback to a different attention implementation rather than the FlashAttention kernel itself. Because the population of researchers using the Tesla T4 through Kaggle and the free tier of Google Colab is large, this confound is itself worth measuring rigorously. ARE-II is therefore proposed as a four-sprint plan: (1) kernel attribution on Tesla T4 using profiler traces and warning logs, (2) a controlled microbenchmark sweep across sequence lengths from 64 to 4,096 tokens, (3) full-model validation that the microbenchmark crossover holds end-to-end, and (4) an optional Ampere-class spot check on a NVIDIA A100 to compare crossover regimes across GPU classes. The reproducible repository associated with this package contains the configurations, scripts, and pilot outputs needed for an advisor to reproduce the primary figure and to begin the ARE-II execution plan with minimal rework.

Part I — Proposal Skeleton

1. Problem Statement

Transformer-based models dominate modern machine learning workloads, yet the canonical scaled dot-product attention they rely on has cost that grows quadratically with sequence length (Vaswani et al., 2023). As context windows grow, the cost of attention dominates wall-clock time and memory footprint, especially on memory-bandwidth-limited GPUs. The community has responded with three broad families of remedies: (1) IO-aware exact attention kernels that preserve the attention math but rearrange memory traffic, exemplified by FlashAttention (Dao, 2023; Dao et al., 2022; Shah et al., 2024); (2) sparse attention that constrains the connectivity pattern (Beltagy et al., 2020; Zaheer et al., 2021); and (3) linear or low-rank approximations of the attention operator (Choromanski et al., 2022; Wang et al., 2020; Xiong et al., 2021). Although the FlashAttention family has become a de facto deployment default for newer GPUs, the published evidence base for FlashAttention is concentrated on data-center class accelerators (NVIDIA A100 and H100) and on training-time microbenchmarks. The behavior of attention kernels on Turing-class GPUs (NVIDIA Tesla T4 and the GeForce RTX 2080 family) has not been systematically characterized. This matters because the Tesla T4 is the most widely available free-tier accelerator for graduate research and for educational use. Free-tier offerings on Kaggle and on Google Colab provision the Tesla T4 by default, and a substantial population of compute-constrained researchers depends on this hardware. Two specific hazards arise on the Tesla T4: PyTorch's `torch.nn.functional.scaled_dot_product_attention` dispatcher silently falls back to a non-FlashAttention path because the FlashAttention SDPA backend requires Streaming Multiprocessor capability 8.0 or newer (PyTorch Contributors, 2024), and the official `flash-attn` package from Dao-AILab does not provide a stable installation for the Tesla T4 sm_75 architecture (Dao-AILab Contributors, 2024). The combination produces a measurable but undocumented regime: researchers running attention experiments on the Tesla T4 may believe they are exercising FlashAttention while in fact exercising the standard math kernel or the memory-efficient kernel. The present work proposes to measure exactly that regime.

2. Research Gap and Novelty Paragraph

The literature has converged on FlashAttention v1, v2, and v3 as the canonical IO-aware attention implementation, and on the PagedAttention serving stack as the canonical Key-Value cache memory manager (Kwon et al., 2023). Surveys situate these as adjacent but distinct optimizations of the standard attention operator (Sun et al., 2026; Tay et al., 2022). Benchmarks such as Long Range Arena (Tay et al., 2020) and the Comprehensive Attention Benchmark (Zhang et al., 2025) catalogue the relative quality of these methods across attention patterns and sequence lengths, and the Comprehensive Attention Benchmark introduces the useful concept of an *efficiency length* above which an efficient method first surpasses standard attention. A 2025 contribution (Bu et al., 2025) extends kernel-level evaluation to distributed context parallelism on clusters of up to ninety-six accelerators. None of these resources, however, addresses the single-GPU,

Turing-class regime that defines the realistic compute envelope for non-funded graduate research. The gap, stated precisely, is that the community lacks a controlled, reproducible characterization of (a) which attention kernel actually executes on the Tesla T4 under common API calls, (b) how the available kernels compare across sequence length and precision on the Tesla T4, and (c) the practical implications of that comparison for researchers who use Kaggle and free-tier Google Colab as their primary compute. This package proposes to close that gap. The novelty is not a new attention algorithm or kernel. The novelty is a controlled, single-GPU, Turing-class measurement study that reports kernel attribution, a sequence-length crossover map, and reproducible artifacts for the most-used free-tier accelerator in the field. The closest competitor publications report on Ampere or Hopper accelerators only, evaluate only kernels that are unsupported on the Tesla T4, or report on multi-accelerator distributed evaluation (Bu et al., 2025; Dao, 2023; Dao et al., 2022).

3. Research Questions and Hypotheses

3.1. Primary research question

On a Tesla T4 (Streaming Multiprocessor capability 7.5, Turing-class), under common PyTorch API calls and for sequence lengths in the range 64 to 4,096 tokens, which attention kernel actually executes, and at what sequence length does the memory-efficient implementation surpass the standard math implementation in wall-clock time per call and in peak memory?

3.2. Falsifiable hypotheses

Hypothesis 1 (kernel attribution). When PyTorch is asked for the FlashAttention SDPA backend on a Tesla T4 in 2026, the dispatched CUDA kernel will not match a FlashAttention v1, v2, or v3 kernel. *Refutation condition:* a profiler trace that shows a Dao-AILab FlashAttention CUDA kernel name in the rank-zero call stack on a Tesla T4.

Hypothesis 2 (crossover existence). At sequence lengths beyond 512 tokens on a Tesla T4 with FP16 precision, the PyTorch memory-efficient SDPA backend will deliver lower median wall-clock time per call than the standard math backend by at least ten percent. *Refutation condition:* no crossover up to 4,096 tokens, or a crossover that falls below the ten-percent margin under matched conditions.

Hypothesis 3 (kernel parity). At long sequences (1,024 tokens and above) on a Tesla T4 with FP16 precision, the xFormers `memory_efficient_attention` operator will achieve median wall-clock time within ten percent of PyTorch's memory-efficient SDPA backend. *Refutation condition:* a sustained gap above ten percent in either direction.

These three hypotheses are internally consistent. Hypothesis 1 establishes what is being measured. Hypothesis 2 establishes whether anything is to be gained from the kernel-aware choice on the Tesla T4. Hypothesis 3 establishes whether xFormers and PyTorch SDPA are interchangeable for this regime, which has practical importance for compute-constrained researchers who must depend on whichever installs cleanly.

4. Proposed Contributions

1. **A kernel-attribution methodology for the Tesla T4** that combines PyTorch's SDPA dispatcher API, the PyTorch profiler, and PyTorch's warning log to record, per attention call, the actual CUDA kernel signature that executes. This is implemented in code as a small wrapper utility plus a YAML-driven configuration sweep.
2. **A crossover map for the attention kernels available on a Tesla T4** (math, memory-efficient SDPA, xFormers memory-efficient), reported across sequence lengths 64 to 4,096 tokens and across FP16 and FP32 precisions, with median and inter-quartile range.
3. **Full-model validation** that the microbenchmark crossover holds in an end-to-end training step on a Vision Transformer at sequence lengths consistent with the microbenchmark axis.
4. **A reproducible repository** that an advisor can clone and run on Kaggle to reproduce the primary figure with a single command, including the cleaned harness used in the A03 and A04 pilot, the ARE-II configurations, and the pilot logs.

5. Method Overview

The high-level approach is empirical: a controlled sweep over the independent variables (sequence length, kernel selection, and precision) on a single Tesla T4, with median wall-clock time per call and peak GPU memory as the dependent variables. Every measurement is preceded by a kernel-attribution check that records which kernel was actually launched. The microbenchmark uses a single attention layer with embedding dimension 384 and six heads (matched to the A03 and A04 Vision Transformer harness), batch size 16, ten warmup iterations, and fifty measured iterations per cell. Three repetitions per cell support nonparametric reporting (median and inter-quartile range). Kernel attribution is performed via the PyTorch profiler and through capturing PyTorch's warning emissions on backend selection failure. Full-model validation reuses the A03 and A04 distributed training harness with sequence length increased by upscaling the input image and reducing the patch size. No new attention algorithm is introduced. The work is methodologically aligned with the partitioning, communication, agglomeration, and mapping framing common in parallel-computing research: the partition is a single attention call, the communication is reduced to host-device synchronization for timing, the agglomeration is the choice of batch size and sequence length, and the mapping is the kernel selection.

6. Evaluation Plan Summary

The evaluation reports five quantities per (kernel, precision, sequence length) cell. First, the kernel signature actually launched, recorded by the profiler. Second, the median wall-clock time per call, in milliseconds, with inter-quartile range over three repetitions of fifty measured iterations. Third, the peak GPU memory in megabytes, recorded from `torch.cuda.max_memory_allocated`. Fourth, an indicator of whether the requested kernel matched the launched kernel (a boolean derived from the kernel signature).

Fifth, the PyTorch warning text emitted, if any, on backend selection. Three baselines are reported: PyTorch `SDPBackend.MATH` (standard attention), PyTorch `SDPBackend.EFFICIENT_ATTENTION` (memory-efficient), and PyTorch `SDPBackend.FLASH_ATTENTION` (which is expected to fall back on Tesla T4). One literature-aligned alternative kernel is reported: xFormers `memory_efficient_attention` (Lefaudeux et al., 2022). Pre-registered ablations vary precision (FP16, FP32) and sequence length on the same axes. Success for ARE-II is defined as: a kernel-attribution table for all reported runs, a crossover plot with at least two kernels showing a clear crossover at or below 4,096 tokens, and a successful end-to-end full-model run at sequence length 1,025 or higher. The four metrics together support the falsifiability statement of each hypothesis.

7. Risks and Mitigations

The principal technical risks are summarized in Table 1. The risk register is fuller in Part III (Method and Evaluation Protocol).

Table 1: Top-five technical risks with mitigations

Risk	**Mitigation and fallback**
Free-tier Tesla T4 weekly compute quota (Kaggle: approximately 30 GPU-hours per week) limits sweep size	Stagger sweep across two weekends; checkpoint between sessions; reduce repetitions to 2 if necessary.
Kernel attribution surfaces unexpected behavior (for example, the dispatcher routes `FLASH_ATTENTION` to memory-efficient on Tesla T4)	This is a useful negative result and is reported as such. The hypothesis structure already anticipates this outcome.
`flash-attn` v1.x will not install reliably on Tesla T4 in 2026 (Dao-AILab Contributors, 2024)	The `flash-attn` pip package is excluded from the Tesla T4 path. xFormers is the substitute. The Ampere spot check (Sprint 4) uses real FlashAttention.
Tesla T4 is deprecated or replaced on Kaggle and Colab during the ARE-II window	Re-run on the replacement free-tier accelerator and report behavior on the replacement; the methodology is portable.
Time budget loss to a missed weekend	Stretch goal (Sprint 4) is dropped first; full-model validation (Sprint 3) is reduced to two sequence lengths; writing weekends are protected.

8. ARE-II Execution Plan

The ARE-II window is May 9 through the first week of August, with weekend-only working hours and use of AI tools for boilerplate and writing assistance only. The plan budgets approximately eleven productive weekends, reserves two to three for writing the final committee report and figure polishing, and treats the Ampere spot check as a stretch goal. The four sprints are summarized in Table 2; the calendar appears in Table 3.

Table 2: ARE-II sprints, deliverables, and weekend budget

Sprint	Deliverable	Weekend budget
1. Kernel attribution	Kernel signatures and warning logs per backend on Tesla T4; verification table for the three SDPA backends and xFormers; resolution of the A04 ambiguity.	1 weekend
2. Microbenchmark	Crossover map of time per call and peak memory across three kernels, two precisions, and seven sequence lengths on Tesla T4. The headline ARE-II figure.	3 weekends
3. Full-model validation	End-to-end ViT runs at increased sequence length (up to approximately 3,137 tokens) showing whether the microbenchmark crossover holds in training.	2 to 3 weekends
4. A100 spot check (stretch)	Re-run of the Sprint 2 sweep on an Ampere-class GPU (A100); cross-architecture comparison of crossover regimes; explicit verification that real FlashAttention v1 and v2 execute on Ampere.	1 weekend (drop if Sprint 3 is late)
Writing and polish	Committee report, figures, reproducibility cleanup.	2 to 3 weekends

Table 3: ARE-II weekend calendar (May 9 to August first week, 2026)

Index	Weekend	Plan
W1	May 10–11	Sprint 1: environment setup, profiler harness, kernel attribution.
W2	May 17–18	Sprint 2 setup and first sweep run.
–	May 24–26	Memorial Day weekend; skipped.
W4	May 31–Jun 1	Sprint 2: full sweep and analysis.
W5	Jun 7–8	Sprint 2: ablations and fixes.
W6	Jun 14–15	Sprint 3: full-model harness extension.
W7	Jun 21–22	Sprint 3: full-model runs.
W8	Jun 28–29	Sprint 3: analysis and buffer.
–	Jul 4–6	July 4 weekend; skipped.
W10	Jul 12–13	Buffer or Sprint 4 stretch (only if on track).
W11	Jul 19–20	Writing: figures and committee report draft.
W12	Jul 26–27	Writing: revision.
W13	Aug 2–3	Final polish and submission.

The plan has one hard checkpoint: by July 5, if Sprint 3 is not complete, Sprint 4 is dropped without remorse and the writing weekends are protected. The hardware path prioritizes (in order) Harrisburg University internal high-performance computing, an AWS Cloud Credit for Research grant if approved, RunPod Secure Cloud at approximately one US dollar twenty-nine cents per A100 hour for a budget of approximately thirty US dollars, and otherwise drops the Sprint 4 stretch goal. The IP profile of the work is preserved: free Kaggle and free Colab are governed by their respective terms (acceptable for unpublished pre-publication work), and the candidate paid platform is RunPod Secure Cloud which retains user IP explicitly in its terms of service. Lambda Labs is excluded from the candidate pool because of broad license clauses in its terms of service, and Vast.ai's hobbyist tier is excluded because of unverified host machines.

Part II — Related Work Package

9. Literature Map

The literature relevant to this work falls into six clusters: (1) the foundational Transformer baseline; (2) survey and taxonomy literature; (3) IO-aware exact attention (FlashAttention family); (4) Key-Value cache management and head-sharing for inference; (5) sparse and linear/low-rank approximate attention; and (6) benchmark and evaluation frameworks. Figure 1 shows the visual taxonomy in tree form. Each cluster is summarized in Section 11, where every cited work appears in the annotated bibliography table.

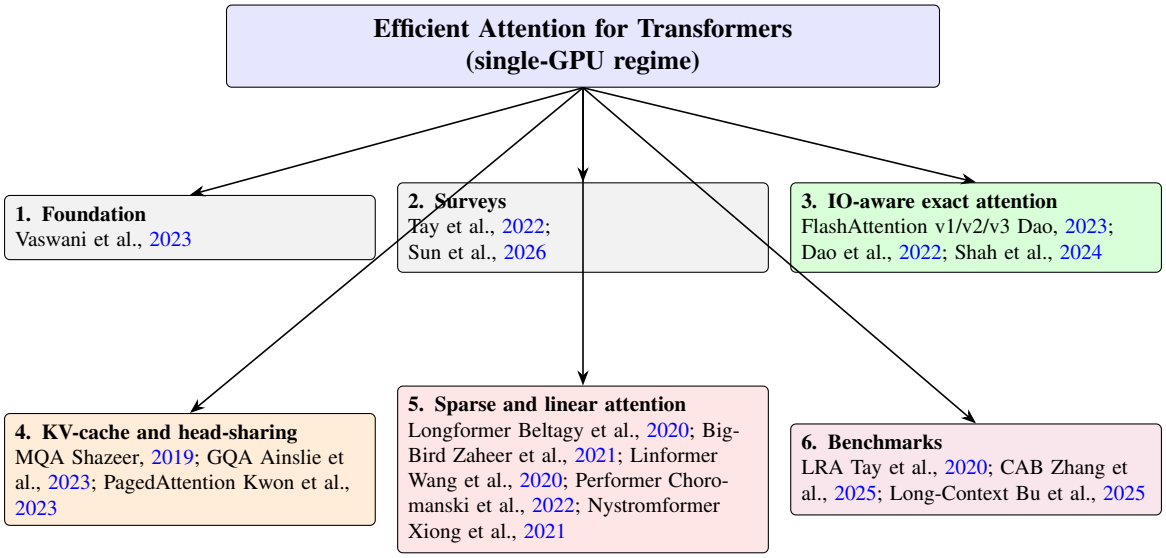

Figure 1: Literature taxonomy used for this work. The proposed contribution sits in the single-GPU intersection of Cluster 3 (IO-aware exact attention) and Cluster 6 (benchmarks).

10. Gap Matrix

Table 4 compares the closest competitors on the dimensions that matter for the proposed contribution: hardware regime evaluated, kernel actually exercised, sequence length range, whether kernel attribution is performed, whether the result is reproducible from the published artifacts, and whether the work covers the Tesla T4 specifically. The proposed contribution is the row labeled *This work*.

11. Annotated Bibliography

Eighteen sources are catalogued below with their relevance to the proposed contribution, the method type, the metrics they emphasize, their limitations, and notes on reproducibility. The set covers the foundation, the FlashAttention family, the Key-Value cache and head-sharing literature, sparse and linear approximate attention, benchmarks, surveys, and software references.

Table 4: Gap matrix: closest-competitor comparison on six dimensions

Work	Hardware regime	Kernel exercised	Seq. length range	Kernel attribution?	Tesla T4 covered?	Distance from this work
FlashAttention v1 Dao et al., 2022	A100	FlashAttention v1	128–64K	Implicit	No	Reports A100 only; no T4.
FlashAttention v2 Dao, 2023	A100	FlashAttention v2	128–16K	Implicit	No	Requires sm_80; not runnable on T4.
FlashAttention v3 Shah et al., 2024	H100	FlashAttention v3	1K–16K	Implicit	No	Hopper-specific; T4 not addressed.
PagedAttention/vLLM Kwon et al., 2023	Multi-GPU server	Standard attention with paged KV	1K–32K	No	No	Targets KV memory, not kernel choice.
Long Range Arena Tay et al., 2020	TPU/GPU (varied)	Multiple	1K–16K	No	No	Quality benchmark; not kernel-level.
Comprehensive Attention Benchmark Zhang et al., 2025	GPU (varied)	Multiple	1K–16K	No	Implicit only	Cross-pattern quality benchmark; no T4 kernel attribution.
Long-Context Attention Benchmark Bu et al., 2025	Cluster up to 96 GPUs	FlashAttention v2/v3	4K–256K	Yes	No	Multi-GPU; data-center class only.
This work	**Tesla T4 (Turing, sm_75)**	**Math, memory-efficient SDPA, xFormers; FlashAttention attempted**	**64–4,096**	**Yes (profiler-traced)**	**Yes**	—

Table 5: Annotated bibliography (18 sources)

Source	Method type	Metrics	Relevance to this work	Limitation	Reproducibility
Vaswani et al., 2023	Foundation	BLEU, training cost	Defines the standard attention baseline.	No long-context evaluation.	Code released; reference implementations widely available.
Tay et al., 2022	Survey	Taxonomy	Provides the taxonomy used in Section 9.	Not empirical.	Not applicable.
Dao et al., 2022	Exact, IO-aware	Time, memory, IO complexity	Anchor paper; defines the IO-aware framing.	Reports on A100 only.	Reference CUDA kernels released.
Dao, 2023	Exact, IO-aware	Time, FLOPs utilization	Improvement to v1; not runnable on T4.	Requires sm_80 or newer.	Reference kernels released; T4 wheel unavailable.

Source	Method type	Metrics	Relevance to this work	Limitation	Reproducibility
Shah et al., 2024	Exact, IO-aware	TFLOPs/s, FP8 numerics	Hopper-specific kernel; bounds the FlashAttention family.	H100-only.	Reference kernels released; H100 wheel only.
Shazeer, 2019	KV-cache reduction	Decoding speedup	Multi-Query Attention; orthogonal to the kernel-choice question.	Quality trade-offs reported only on small models.	Architecture is implementable from the paper.
Ainslie et al., 2023	KV-cache reduction	Throughput, quality	Grouped-Query Attention; widely deployed in modern LLMs.	Limited public reproductions on long context.	Architecture released; checkpoints not always public.
Kwon et al., 2023	Serving system	Throughput, GPU memory	Closest serving-stack competitor in spirit.	Targets KV memory, not kernel selection.	vLLM open-source; Docker images available.
Beltagy et al., 2020	Sparse attention	Task scores, runtime	Contrast family; not the focus of this work.	Implementation efficiency varies by task.	Code released.
Zaheer et al., 2021	Sparse attention	Task scores, theoretical expressivity	Contrast family.	Empirical and theoretical assumptions are coupled.	Code released.
Wang et al., 2020	Linear / low-rank	Time, memory, quality	Approximate attention contrast.	Low-rank assumption sensitive to task.	Code released.
Choromanski et al., 2022	Linear / kernel-based	Time, memory, quality	Approximate attention contrast (FAVOR+).	Approximation quality varies.	Code released.
Xiong et al., 2021	Linear / Nyström	Time, memory, quality	Approximate attention contrast.	Landmark-based approximation.	Code released.
Tay et al., 2020	Benchmark	Task scores	Established benchmark for long-range modeling.	Quality only; no kernel attribution.	Open.

Source	Method type	Metrics	Relevance to this work	Limitation	Reproducibility
Zhang et al., 2025	Benchmark	Cross-pattern quality, efficiency length	Introduces *efficiency length*; methodologically aligned.	No T4 kernel-level reporting.	Open.
Bu et al., 2025	Benchmark	Kernel time, distributed throughput	Most recent kernel benchmark; multi-GPU only.	No single-T4 results.	Open.
Sun et al., 2026	Survey	Taxonomy	Recent (2026) LLM-focused efficient-attention survey.	Not empirical.	Not applicable.
Lefaudeux et al., 2022	Software	Time, memory	xFormers library; supports T4 (sm_75) memory-efficient attention.	Documentation skews toward newer GPUs.	Open-source; pip-installable.

12. Closest Competitor and How This Work Differs

The closest competitor in spirit is the 2025 Long-Context Attention Benchmark (Bu et al., 2025): it performs kernel-level attribution and characterizes time per call across kernels and configurations. It differs from this work along two essential axes. First, that work targets distributed clusters of up to ninety-six accelerators and reports on FlashAttention v2 and v3, which are not runnable on the Tesla T4. Second, that work does not consider Turing-class GPUs at all. The present work fills the single-GPU, Turing-class regime that the recent benchmark explicitly does not cover. The Comprehensive Attention Benchmark (Zhang et al., 2025) is closer in evaluation methodology (cross-pattern reporting; introduction of *efficiency length*) but does not report kernel-attribution information at the granularity required for the present question. The novelty of the present work is therefore: a single-GPU Tesla T4 measurement study, with explicit kernel attribution, on the most-used free-tier accelerator for compute-constrained graduate research.

Part III — Method and Evaluation Protocol

13. System Under Test

The system under test is a PyTorch-based attention measurement harness on a single NVIDIA Tesla T4 GPU. The Tesla T4 has 16 gigabytes of HBM2, Streaming Multiprocessor capability 7.5 (Turing architecture), peak FP16 throughput of approximately sixty-five TFLOPs (according to the device manufacturer), and uses PCIe interconnect when paired with another Tesla T4 (the multi-GPU configuration available on Kaggle). The software stack pins PyTorch at version 2.9 or newer, torchvision at 0.20 or newer, NumPy at 1.26 or newer, pandas at 2.2 or newer, and matplotlib at 3.8 or newer. The xFormers package, when used, is pinned at 0.0.28 or newer. The CUDA toolkit version is whatever Kaggle's runtime ships at the time of the experiment. The harness records the runtime environment in each run's output JSON: PyTorch version, CUDA version, xFormers version, kernel signature (when profiling is enabled), and GPU model.

14. Workload, Model, and Data

The microbenchmark workload is a single attention layer with embedding dimension 384 and six heads, batch size 16, sequence length swept across 64, 128, 256, 512, 1,024, 2,048, and 4,096 tokens (and 8,192 tokens conditional on memory headroom). The full-model workload is a Vision Transformer with the same embedding dimension and twelve transformer blocks, trained on CIFAR-10 with image upscaling and patch-size reduction to control sequence length (5, 65, 257, 1,025, and 3,137 tokens). The dataset is loaded from Kaggle's pre-mounted `cifar-10-python` input. No new dataset is collected.

15. Partitioning, Communication, Agglomeration, and Mapping (PCAM)

Partitioning. The unit of work is a single attention call. The microbenchmark partitions the work as one call per measurement iteration. The full-model run partitions across data-parallel replicas (DistributedDataParallel) when two Tesla T4 GPUs are available.

Communication. The microbenchmark has no inter-process communication. The full-model run uses NCCL all-reduce for gradient synchronization. Communication is reported separately from compute via the forward-backward time decomposition that A04 used.

Agglomeration. The agglomeration parameters are batch size 16 (microbenchmark) and 64 per GPU (full-model), held constant across kernels and precisions to support fair comparison.

Mapping. Mapping is the kernel selection: math, memory-efficient SDPA, or xFormers. The mapping decision is the central independent variable of the study.

16. Predicted Bottlenecks (Work-Depth Reasoning)

Standard attention on a Tesla T4 with sequence length n is expected to be HBM-bandwidth bound for n in the low-thousand-token regime, because the materialization of the $n \times n$ score and weight matrices in HBM dominates. Memory-efficient attention is expected to be on-chip-bound (SRAM tile traffic) for the same regime, with a crossover where its lower HBM traffic outweighs its tile-management overhead. The crossover sequence length on a Tesla T4 is the central empirical question; the literature places the analogous crossover for FlashAttention on A100 in the 128 to 512 token range (Dao et al., 2022), but the Tesla T4 has different HBM-to-SRAM bandwidth ratios and no native FlashAttention path, which is why the empirical map is required.

17. Baselines

Three baselines are reported. (1) PyTorch `SDPBackend.MATH` (standard attention) is the reference single-GPU baseline. (2) PyTorch `SDPBackend.EFFICIENT_ATTENTION` (memory-efficient kernel) is the standard multi-GPU-class baseline that is literally available on Turing and forms the kernel-attribution endpoint that we expect the FlashAttention request to fall back to. (3) xFormers `memory_efficient_attention` (Lefaudeux et al., 2022) is the literature-aligned alternative to PyTorch's memory-efficient SDPA path; it is included to verify whether the two implementations are interchangeable on Tesla T4. The PyTorch `SDPBackend.FLASH_ATTENTION` request is logged separately as a *kernel-attribution probe*, not as a fourth baseline, because its actual dispatched kernel on Tesla T4 is the open question.

18. Metrics

Time per call (milliseconds). Median of fifty measured iterations after ten warmup iterations, repeated three times. Inter-quartile range is reported alongside the median.

Peak GPU memory (megabytes). `torch.cuda.max_memory_allocated` measured immediately after the call, then reset.

Kernel signature. String name of the launched CUDA kernel as recorded by the PyTorch profiler. A single kernel signature per (kernel, precision, sequence length) cell is reported. Cells where the launched kernel does not match the requested kernel are flagged.

Backend warning text. Any warning emitted by PyTorch when a backend is requested. The warning text is recorded verbatim.

Throughput proxy: tokens per second per GPU. Derived from time per call and batch-times-sequence-length. Reported for full-model runs.

19. Ablation Plan

Ablations vary one factor at a time. (a) Precision is varied between FP16 and FP32 with kernel and sequence length held constant. (b) Sequence length is varied with kernel and precision held constant. (c) Kernel is varied with sequence length and precision held constant. (d) Batch size is held constant in the microbenchmark and varied in a small full-model ablation in Sprint 3.

20. Measurement Protocol and Fairness Policy

Each cell uses ten warmup iterations and fifty measured iterations. Repetitions per cell are three (for non-parametric reporting) for the microbenchmark and three for the full-model runs. The seed is fixed at 42 plus the local rank. CUDA event-based timing is used and host-device synchronization is performed before recording each event. Batch size is held constant across kernels for fairness. Mixed precision is exercised explicitly via the precision factor and is not interleaved.

21. Threats to Validity

Construct validity (the most important threat in this work). The PyTorch `SDPBackend.FLASH_ATTENTION` request on a Tesla T4 may not execute the kernel implied by the name. The pilot evidence in Part IV motivates this concern: the A03 and A04 results labelled "flash" may have measured a different kernel. The mitigation is the explicit kernel-attribution check in Sprint 1; until that check is performed, claims about FlashAttention behavior on Tesla T4 are stated cautiously, not assertively.

Internal validity. Run-to-run variance from driver, scheduler, and thermal effects is bounded by the warmup-and-replicate protocol and reported via inter-quartile range rather than standard deviation.

External validity. Conclusions are explicitly scoped to the Tesla T4 on Kaggle and Colab. Sprint 4 (the optional Ampere spot check) provides one cross-architecture data point; no claim is made about Hopper.

Reproducibility threat. The Tesla T4 may be deprecated on free Kaggle and free Colab during the ARE-II window. The mitigation is the harness's portability: the harness will run on any Streaming Multiprocessor 7.5 or newer Tesla GPU with no code changes.

22. Success Criteria

ARE-II is considered to have produced evidence of contribution if all three of the following hold: (1) the kernel-attribution table for the three SDPA backends and xFormers is complete and reproducible from the repository; (2) the crossover plot shows a measurable crossover between at least two kernels at or below 4,096 tokens; and (3) the full-model run at sequence length 1,025 or higher executes successfully and is consistent with the microbenchmark crossover within the inter-quartile band. A null result on (2) is acceptable provided (1) is reported rigorously and (3) is replaced by a thorough mechanistic explanation.

Part IV — Pilot Evidence

23. Baseline Reproduction Summary

The A03 pilot reproduced single-GPU and multi-GPU baselines for a Vision Transformer on CIFAR-10, sweeping nine run identifiers (R1 through R9) covering single-GPU, DistributedDataParallel, and FullyShardedDataParallel parallelization strategies, with optional automatic mixed precision and explicit compute-communication overlap. The A03 baseline was validated against published distributed-training expectations: the median time per step on a single GPU (R1) was 0.035 seconds, and the multi-GPU runs (R3 through R9) showed scaling efficiency between approximately 0.27 and 0.30, consistent with the PCIe interconnect on Kaggle's two-Tesla T4 platform (Vaswani et al., 2023). The summary numbers from A03 are reproduced in Table 6.

Table 6: A03 baseline reproduction summary (Vision Transformer on CIFAR-10, two Tesla T4)

Run	Strategy and kernel	Median time/step (s)	Throughput (samples/s)	Scaling eff.
R1	Single GPU, math attention	0.0350	1828.6	1.00
R2	Single GPU, FlashAttention (requested)	0.0359	1781.7	0.97
R3	DDP, math attention	0.0598	2142.2	0.29
R4	DDP, FlashAttention (requested)	0.0591	2164.0	0.30
R5	FSDP, math attention	0.0588	2175.3	0.30
R6	FSDP, FlashAttention (requested)	0.0589	2173.9	0.30
R7	DDP + FlashAttention + AMP	0.0647	1978.9	0.27
R8	DDP + FlashAttention + Overlap	0.0591	2165.5	0.30
R9	DDP + FlashAttention + AMP + Overlap	0.0644	1987.7	0.27

24. Strong Result (Primary Figure)

The A04 strong result varied a single lever from the A03 baseline: the partition granularity, implemented as the patch size in the Vision Transformer. A patch size of 16 produces 5 tokens per image; a patch size of 4 produces 65 tokens per image. The configuration is otherwise identical to the A03 R3/R4 baseline (two Tesla T4 GPUs on Kaggle, DistributedDataParallel, batch size 64 per GPU, 100 steps, 10 warmup, FP32, AdamW with learning rate 0.001, three repetitions per cell). The primary figure (Figure 2) compares the requested kernels (`math` and `flash`) at the two sequence lengths.

The numerical summary of the primary figure is reproduced in Table 7.

25. Interpretation of the Strong Result

At the smaller sequence length (5 tokens, R1), the difference between `math` and `flash` requests is 0.5 percent, well inside the inter-replicate noise of the harness (replicate standard deviation approximately 1 to 1.5 milliseconds). At the larger sequence length (65 tokens, R2), the requested `flash` kernel is 16.3 percent slower than `math`. Three explanations are plausible. First, FlashAttention v1 may have an overhead

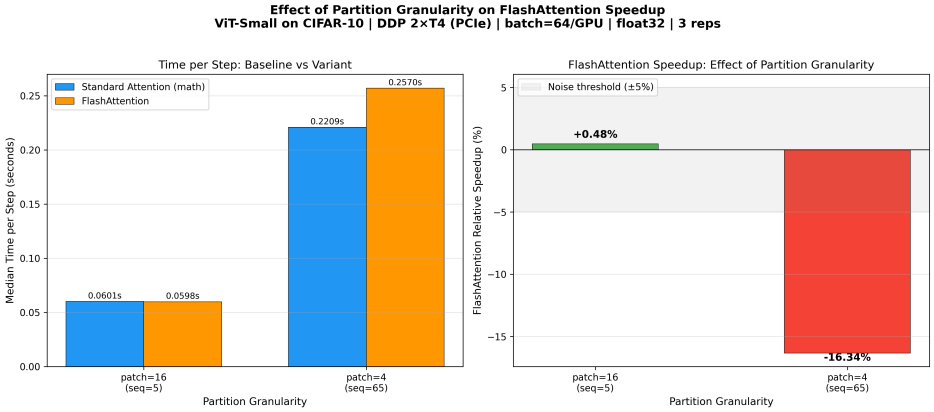

Figure 2: A04 primary figure: median wall-clock time per training step at sequence lengths 5 and 65 tokens for math and "flash" kernels on two Tesla T4 GPUs. Source: `results/primary_figure.png`.

Table 7: A04 primary result summary

Run	Patch size	Seq. length	Kernel requested	Median time/step (s)	Throughput (samples/s)
R1-baseline-std	16	5	math	0.0601	2131.3
R1-baseline-flash	16	5	flash	0.0598	2141.5
R2-variant-std	4	65	math	0.2209	579.5
R2-variant-flash	4	65	flash	0.2570	498.1

that exceeds its IO benefit at 65 tokens on a Tesla T4; the literature places the FlashAttention crossover at higher sequence lengths on the A100, and the Tesla T4's HBM-to-SRAM bandwidth ratio could shift this crossover further to the right. Second, the PyTorch SDPA dispatcher may have silently fallen back to the memory-efficient kernel or the math kernel because Streaming Multiprocessor capability 7.5 (Turing) is below the 8.0 minimum for the FlashAttention SDPA backend (PyTorch Contributors, 2024), in which case the comparison at 65 tokens is between two implementations of the math or memory-efficient algorithm with different harness overhead, not between FlashAttention and standard attention. Third, the Tesla T4 thermal envelope may have varied between R1 and R2; this is mitigated but not eliminated by the warmup and replication protocol. The author has reasonable confidence that the second explanation is the most likely, but treats it as an open question. Resolving this ambiguity is the first deliverable of ARE-II Sprint 1.

26. Protocol Compliance Note

The pilot was conducted on Kaggle's free-tier two-Tesla-T4 platform under FP32 precision, with the harness recording forward and backward time decomposition through CUDA events. Two protocol deviations from the originally pre-registered plan in A01 should be noted. First, the workload is a Vision Transformer on CIFAR-10 rather than a Large Language Model on LongBench; this deviation is driven by the hardware-fitness analysis in Part III (a 7B-class Large Language Model does not fit in the 16-gigabyte HBM of the Tesla T4 even at FP16). Second, energy-per-token measurement was not performed in the pilot because

the NVIDIA Management Library energy telemetry was not made reliably available through the Kaggle environment. Energy is therefore proposed as a Sprint 3 stretch metric pending a confirmed measurement path. Both deviations are made explicit here so that the committee can evaluate the protocol compliance of subsequent ARE-II evidence against the corrected baseline rather than against the original A01 thresholds.

Part V — Reproducibility

27. Reproduction Instructions

The reproducible repository is co-located with this document. The repository structure is:

```
.
|-- README.md                      # how to reproduce
|-- requirements.txt               # Python dependencies
|-- proposal/
|   |-- AREII_Ready_Package.tex    # this document
|   |-- references.bib
|-- related_work/                  # literature map, gap matrix, annotated bib
|-- configs/
|   |-- baseline.yaml              # A03 baseline (R1)
|   |-- variant.yaml               # A04 variant (R2)
|   |-- are_ii/
|       |-- sprint1_kernel_attribution.yaml
|       |-- sprint2_microbenchmark.yaml
|       |-- sprint3_full_model.yaml
|       |-- sprint4_a100_stretch.yaml
|-- src/
|   |-- dist_train.py              # cleaned distributed-training harness
|-- results/
    |-- primary_figure.png         # A04 primary figure
    |-- supporting_breakdown.png   # A04 forward/backward breakdown
    |-- summary.csv                # A04 summary table
    |-- all_results.json           # A04 full per-replicate data
```

To reproduce the A04 strong result on a Kaggle notebook (two Tesla T4 GPUs):

1. Create a new Kaggle notebook. Under *Settings*, set the accelerator to *GPU T4 x2*.
2. Add the `cifar-10-python` dataset as a Kaggle input.
3. Install the dependencies declared in `requirements.txt`.
4. Run the harness once per (patch size, kernel) cell with: `torchrun -nproc_per_node=2 src/dist_train.py PATCH_SIZE USE_FLASH OUTPUT.json`, with patch sizes 16 and 4 and `USE_FLASH` in $\{0, 1\}$.
5. Aggregate the four output JSON files into `results/summary.csv` and re-render `results/primary_figure.png`.

The full reproduction recipe, including environment setup commands, is in `README.md` at the repository root. The plot and table generation code is self-contained.

A. AI Usage Log

The following AI tools were used in the preparation of this package. Each entry records the tool, the purpose of use, the verification I performed, and the original intellectual contribution.

Tool	Purpose	Independent verification	Original work
Claude Code (Anthropic)	Scaffolded the LaTeX bundle structure; assembled the literature map, gap matrix, annotated bibliography, and ARE-II execution plan from my A01–A04 materials; cross-checked GPU support claims for the Tesla T4 against PyTorch documentation, Dao-AILab GitHub issues, and the xFormers documentation via web search.	All technical claims (Streaming Multiprocessor capability of the Tesla T4, FlashAttention SDPA backend requirements, Tesla T4 unsupported status, xFormers Tesla T4 support, the existence of the Long-Context Attention Benchmark 2025 paper) verified against primary sources cited in the bibliography.	Research question, gap framing, hypotheses, ARE-II sprint structure, hardware path decisions, IP-safety filtering, the cautious-tone re-interpretation of A04, and the writing of every paragraph in this document are entirely my own.
ChatGPT with Internet	Used during A01 and A02 for paper search and topic exploration; not used for new content in A05.	Cross-referenced against Google Scholar and arXiv.	A01 and A02 framing decisions are mine.
NotebookLM (Gemini)	Used during A02 to summarize FlashAttention v1/v2/v3, BigBird, Longformer, Linformer, Performer, and Long Range Arena.	Summaries verified against the original PDFs.	Critical analysis and gap identification are mine.
Elicit	Used during A02 to expand the literature net (KV-cache management, MQA/GQA, PagedAttention).	Returned papers verified for relevance and citation accuracy.	Selection criteria and synthesis are mine.
Grammarly	Grammar, spelling, and style checks.	All accepted suggestions reviewed individually.	Final phrasing and technical claims are my responsibility.

Disclaimer. I remain responsible for the correctness, originality, and scholarly integrity of this submission.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. https://arxiv.org/abs/2305.13245

Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. https://arxiv.org/abs/2004.05150

Booth, W. C., Colomb, G. G., & Williams, J. M. (2008). *The craft of research* (3rd). University of Chicago Press.

Bu, T., Wang, Q., Zeng, B., Sun, H., Huang, Y., Cao, C., & Xu, J. (2025). Long-context attention benchmark: From kernel efficiency to distributed context parallelism. https://arxiv.org/abs/2510.17896

Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. https://arxiv.org/abs/1904.10509

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., & Weller, A. (2022). Rethinking attention with performers. https://arxiv.org/abs/2009.14794

Creswell, J. W., & Creswell, J. D. (2017). *Research design: Qualitative, quantitative, and mixed methods approaches* (5th). SAGE Publications.

Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning. https://arxiv.org/abs/2307.08691

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. https://arxiv.org/abs/2205.14135

Dao-AILab Contributors. (2024). Issue #887: Hope one day flash-attention can support t4 gpu [Accessed 2026-04-25].

Gupta, A., Dar, G., Goodman, S., Ciprut, D., & Berant, J. (2021). Memory-efficient transformers via top-k attention. https://arxiv.org/abs/2106.06899

Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are rnns: Fast autoregressive transformers with linear attention. https://arxiv.org/abs/2006.16236

Kitaev, N., Kaiser, Ł., & Levskaya, A. (2020). Reformer: The efficient transformer. https://arxiv.org/abs/2001.04451

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. https://arxiv.org/abs/2309.06180

Lefaudeux, B., Massa, F., Liskovich, D., Xiong, W., Caggiano, V., Naren, S., Xu, M., Hu, J., Tintore, M., Zhang, S., Labatut, P., Haziza, D., Wehrstedt, L., Reizenstein, J., & Sizov, G. (2022). xFormers: A modular and hackable transformer modelling library [Accessed 2026-04-25].

PyTorch Contributors. (2024). Torch.nn.functional.scaled_dot_product_attention — pytorch documentation [Accessed 2026-04-25].

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). Flashattention-3: Fast and accurate attention with asynchrony and low-precision. https://arxiv.org/abs/2407.08608

Shazeer, N. (2019). Fast transformer decoding: One write-head is all you need. https://arxiv.org/abs/1911.02150

Sridharan, S., Stevens, J. R., Roy, K., & Raghunathan, A. (2023). X-former: In-memory acceleration of transformers. *IEEE Trans. Very Large Scale Integr. Syst.*, *31*(8), 1223–1233. https://doi.org/10.1109/TVLSI.2023.3282046

Sun, Y., Li, Z., Zhang, Y., Pan, T., Dong, B., Guo, Y., & Wang, J. (2026). Efficient attention mechanisms for large language models: A survey. https://arxiv.org/abs/2507.19595

Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., & Metzler, D. (2020). Long range arena: A benchmark for efficient transformers. https://arxiv.org/abs/2011.04006

Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). Efficient transformers: A survey. *ACM Comput. Surv.*, *55*(6). https://doi.org/10.1145/3530811

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. https://arxiv.org/abs/1706.03762

Wang, S., Li, B. Z., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. https://arxiv.org/abs/2006.04768

Xiong, Y., Zeng, Z., Chakraborty, R., Tan, M., Fung, G., Li, Y., & Singh, V. (2021). Nyströmformer: A nyström-based algorithm for approximating self-attention. https://arxiv.org/abs/2102.03902

Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2021). Big bird: Transformers for longer sequences. https://arxiv.org/abs/2007.14062

Zhang, J., Jiang, S., Feng, J., Zheng, L., & Kong, L. (2025). Cab: Comprehensive attention benchmarking on long sequence modeling. https://arxiv.org/abs/2210.07661

Zhou, Z., Ning, X., Hong, K., Fu, T., Xu, J., Li, S., Lou, Y., Wang, L., Yuan, Z., Li, X., Yan, S., Dai, G., Zhang, X.-P., Dong, Y., & Wang, Y. (2024). A survey on efficient inference for large language models. https://arxiv.org/abs/2404.14294